(There's also exact_text, which just takes a string and requires the string to match the text of the selector exactly).

- visible. By default, Capybara only matches visible elements, but you can change that behavior with visible: :hidden, or visible: :all. (The latter finds all elements regardless of visible state.)

You can augment have_selector in a number of ways. The selector argument to have_selector can be just an element, as in div, or an element plus a class or ID decoration, as in div.hidden. In the latter case, a matching element must have both the HTML tag and the DOM class or ID. As with other DOM selectors, a dot (.) indicates a DOM class and a hash mark (#) indicates a DOM ID. You can also use brackets to indicate arbitrary HTML attributes, as in input[name='email']. The Capybara docs have more details; in particular, see the description of the all method.[4]

If you want to specify that a given selector does not exist on the page, in RSpec you can use either not_to have_selector or to have_no_selector, which are equivalent. In Minitest, though, you must use assert_no_selector.

Our test first uses have_selector to validate that .name and .size elements exist and match any options given. Capybara has some other methods you might use that are less powerful or flexible than have_selector, such as have_text and has_link?.

Our test also uses the within method, which takes a selector argument and a block. Capybara expects the selector passed to within to match a single element on the page. Inside the block argument, any Capybara call is scoped to only find or assert against the contents of that element. So, the part of your test that looks like this:

```
within("#task_2") do
  expect(page).to have_selector(".name", text: "Find UFOs")
end
```

will pass only if there is a .name element inside the #task_2 element. This can also be written as expect(page).to have_selector("#task_2 .name", text: "Cast the designers"), though I find that form a little flaky and hard to read.

Finally, the most useful Capybara method when things go wrong is save_and_open_page, which dumps the contents of the Capybara DOM into a temp file and opens the temp file in a browser (opening in a browser requires a gem called Launchy). You won't have any CSS or images with relative file

4. http://rubydoc.info/github/jnicklas/capybara/master/Capybara/Node/Finders#all-instance_method